

Astrophysical Recipes

The Art of AMUSE

Simon Portegies Zwart

Sterrewacht Leiden
Leiden University
Einsteinweg 55
2333 CC, Leiden
the Netherlands

Steve McMillan

Department of Physics
Drexel University
Philadelphia, PA 19104, USA

Steven Rieder

Anton Pannekoek Instituut
Science Park 904
1098 XH Amsterdam
the Netherlands

June 13, 2025

In memoriam



In memoriam Inti Pelupessy
Gallo 1977 - Leiden 2024

This photograph shows Inti Pelupessy at the Archeon Museum Park in the Netherlands, trying to make fire by rubbing together two wooden sticks. Inti was strongly involved from the start in the AMUSE project. He passed away in the summer of 2024, and is missed by all.

As a brilliant hydrodynamics expert (trained by the very best), Inti quickly became a key figure in the development of AMUSE, his swift mastery of this complex framework is a testament to his remarkable intellect. His pioneering work in this area resulted in important contributions to the field. Without his outstanding qualities as a researcher, his sharp thinking, his excellent understanding of the physics and his unsurpassable programming skills, AMUSE would simply not exist.

Possibly even more striking were his social skills. As you can see on the photograph, Inti was a natural educator, teaching on subjects ranging from procedural operations to thermodynamics, here demonstrated in the making of fire. Inti was reluctant to promote his own work, and always listened to the opinions of others before elaborating on his own ideas. Still, he was often one of the strongest minds in a collaboration.

Those of us fortunate enough to have known Inti will remember his deep insight, kindness, patience, and his dedication to science. His loss is felt deeply by us, and by all in his periphery but his legacy in both science and mentorship will live on.

Contents

Preface	1
Acknowledgments	5
I Computational Astrophysics	7
1 What is Computational Astrophysics?	8
1.1 Computational astrophysics	8
1.1.1 Origin of this book	8
1.1.2 Hands on <i>is</i> hands on	9
1.1.3 What about the math?	9
1.1.4 Alternative simulation environments	10
1.1.5 Objective of this book	11
1.1.6 What is missing from this book	12
1.1.7 Outline of the book	12
1.2 A brief history of simulations in astrophysics	13
1.2.1 The first simulation experiments	15
1.3 Software used in this book	17
1.3.1 Motivation for a homogeneous software environment	17
1.3.2 Choice of programming languages	18
1.3.3 Design of AMUSE	19
1.3.4 Terminology	21
1.4 Installing AMUSE	22
1.4.1 Installing AMUSE in a virtual environment	23
1.4.2 Running the examples	24
1.4.3 Running AMUSE	24
2 Fundamental concepts	29
2.1 Programming	29
2.1.1 Why Python	29
2.1.2 Style guide	30
2.2 Internal data handling and communication	31
2.2.1 Particles and Particle sets	32
2.2.2 Grid-based data	34

2.2.3	Data handling between code and script	37
2.2.4	Storing and Recovering Data	38
2.3	Converting between code units and SI	39
2.3.1	Scaling standard units to internal code units	39
2.3.2	Alternative converters	40
2.3.3	Changing Parameters of the simulation	41
2.4	Tricks with units	42
2.4.1	Setting printing strategies	42
2.4.2	The command-line argument parser	43
3	Initial conditions	46
3.1	Starting simulations	46
3.1.1	Warning on initial conditions	46
3.1.2	Variations in starting points	48
3.2	Creating particle distributions in space	49
3.2.1	Initial condition for gravitational dynamics	49
3.2.2	Initial condition for hydrodynamics	53
3.3	Creating mass distributions	55
3.4	Creating systems of multiple stars and planets	58
3.4.1	The Solar system	58
3.4.2	Other planetary systems	62
3.4.3	Binaries and hierarchical multiples	62
3.5	Often used initial conditions	64
3.6	Assignments	65
3.6.1	Create your own initial conditions generator or manipulator	65
3.6.2	Solar system minor bodies	65
II	Fundamental principles	67
4	Gravitational Dynamics	68
4.1	In a Nutshell	68
4.1.1	Equations of Motion for a Self-gravitating System	68
4.1.2	Gravitational time scales	69
4.1.3	Star Cluster Dynamics	73
4.1.4	Physics of the Integrator	77
4.2	N -body Integration Strategies	79
4.2.1	Global Structure of an N -body Code	79
4.2.2	Types of N -body Code	80
4.2.3	Discretization Strategies in N -body Simulations	82
4.3	The AMUSE interface to Gravity Solvers	85
4.3.1	Generating Initial Conditions	88
4.3.2	Specifying and initializing the gravity solver	88
4.3.3	Feeding Particles to the N -body Code	89
4.3.4	Evolving the Model to the Desired Time	90

4.3.5	Scaling the simulation	91
4.3.6	Code parameters	91
4.3.7	Changing the solver used in the simulation	92
4.4	Examples	93
4.4.1	Integrating the Orbits of Venus and Earth	93
4.4.2	Relativistic N-body integration	100
4.4.3	Fighting exponential divergence	100
4.4.4	Secular Multiples	102
4.4.5	Merging Galaxies	106
4.5	Choosing the right code	108
4.5.1	Stopping conditions	108
4.6	Validation	110
4.6.1	Error Propagation and Validation	110
4.7	Assignments	113
4.7.1	Orbital Trajectories	113
4.7.2	Vostok	113
4.7.3	Dynamical Binary Formation	113
4.7.4	L_1 Lagrangian Point	114
4.7.5	Virial Equilibrium	115
5	Stellar Structure and Evolution	124
5.1	In a Nutshell	124
5.1.1	Stellar Time Scales	127
5.1.2	Physics of the Interior	129
5.1.3	Final Stages of Stellar Evolution	134
5.2	Simulating Stellar Evolution	136
5.2.1	Initial conditions for Stars	137
5.2.2	Stellar Evolution Modules in AMUSE	138
5.2.3	Changing the metallicity of a star	141
5.2.4	Starting a star on the pre-main-sequence	141
5.2.5	Improving the Stellar Evolution Solver	142
5.2.6	Evolving an Inhomogeneous Stellar Population	143
5.2.7	Enforcing Stellar Mass Loss/Gain	144
5.2.8	Accessing Stellar Interiors	146
5.2.9	Modeling Stellar Mergers	147
5.2.10	Interrupting Stellar Evolution	149
5.3	Binary Evolution	150
5.3.1	Stellar winds	150
5.3.2	Tidal circularization and synchronization	150
5.3.3	Roche lobe	151
5.3.4	Stable mass transfer	152
5.3.5	Common-envelope evolution	152
5.3.6	Supernova explosions	153
5.3.7	Magnetic braking	153
5.3.8	Gravitational-wave radiation	153

5.3.9	Initializing and Evolving a Binary	154
5.3.10	Initial binary fraction	156
5.3.11	Initial distribution functions for binaries	156
5.3.12	Rejuvenation Due to Collision or Accretion	157
5.3.13	Reading and Writing Binary Evolution Files	158
5.4	Examples	158
5.4.1	Response of a Star to Mass Loss	158
5.4.2	Blue Stragglers in M67	159
5.5	Validation	161
5.6	Assignments	163
5.6.1	Stellar Comparison	163
5.6.2	Ages of the M67 Blue Stragglers	164
5.6.3	Constructing Isochrones	164
6	Hydrodynamics	171
6.1	In a Nutshell	171
6.1.1	Underlying Equations	172
6.1.2	Turbulence and Shocks	175
6.1.3	Hydrodynamical Time Scales	177
6.1.4	Hydrodynamical Instabilities	178
6.1.5	Physics of the Integrator	181
6.2	Hydrodynamics in AMUSE	182
6.2.1	Types of Hydrodynamics Code	182
6.2.2	Smoothed Particle Hydrodynamics	185
6.2.3	Grid-based Methods	186
6.2.4	Managing Shocks and Discontinuities	189
6.2.5	Initializing a Grid from a Particle Distribution	189
6.3	Extending the Hydrodynamics Solver	191
6.3.1	Generating New SPH Particles	192
6.3.2	Removing SPH Particles	193
6.3.3	Sinks and Dark Matter Particles	194
6.3.4	Cooling the Gas Using an External Routine	196
6.3.5	Stopping conditions	196
6.4	Some simple experiments	197
6.4.1	Circumstellar Disk with a Bump	197
6.4.2	Colliding Stars	199
6.4.3	Accreting from the Wind of a Companion	201
6.4.4	Continuing with Hydrodynamics after a Henyey Code Crash	203
6.5	Collapsing Molecular Cloud	204
6.5.1	Star Formation in a Collapsing Molecular Cloud	206
6.5.2	Stellar Winds and spherical outflows	207
6.5.3	The Collapse Calculation	208
6.5.4	Collapsing Molecular Cloud in grid-based hydrodynamics	211
6.6	Validation	212
6.6.1	Riemann Shock Tube Problem	212

6.6.2	Kelvin–Helmholtz Test	213
6.6.3	Cloud-shock Test	214
6.6.4	Boss–Bodenheimer Test	215
6.7	Assignments	216
6.7.1	Convergence Test	216
6.7.2	Testing the Boss–Bodenheimer Test	218
6.7.3	The Dissolving Bump	218
6.7.4	Collapsing Molecular Cloud with Sink Particles	218
6.7.5	A Star-forming Region	219
6.7.6	Neutron Star Hits Companion	220
6.7.7	Supernova Explosion	220
7	Radiative Transfer	228
7.1	In a Nutshell	228
7.1.1	Underlying Equations	229
7.1.2	Physics of the Integrator	232
7.2	Radiative Transfer in AMUSE	233
7.2.1	Radiative Transfer Modules	233
7.2.2	Ionization of a Molecular Cloud	236
7.3	Examples	240
7.3.1	Ionization Front in an H ₂ Region	240
7.4	Validation	244
7.5	Assignments	247
7.5.1	Habitability of a protoplanetary disk	247
7.5.2	Bumpy disks	248
III	Advanced strategies	251
8	Elementary Coupling Strategies	252
8.1	Multi-code coupling problems	252
8.2	Advanced data handling	253
8.2.1	Dedicated channels and copy operations	253
8.2.2	Particle subsets and supersets	257
8.2.3	Interrupting codes and stopping conditions	258
8.2.4	Collision handling	260
8.3	Runtime analysis and task offloading	263
8.3.1	The <code>Hop</code> package	263
8.3.2	The <code>Kepler</code> package	264
8.3.3	Recovering from a code crash	265
8.4	Elementary code coupling strategies	266
8.4.1	Combining gravity with stellar evolution	266
8.4.2	Coupling Radiative Transfer with Hydrodynamics	268
8.4.3	Switching from one stellar evolution code to another	269
8.4.4	Event-driven simulations	269

8.4.5	Evolution of a hierarchical triple system	273
8.5	Stellar cluster with collisions	277
8.5.1	Sticky stars	277
8.5.2	Using a separate code to manage a collision	277
8.5.3	Restarting stellar evolution after a merger	278
8.5.4	Using a hydro code to simulate a stellar merger	280
8.6	Complex linear code-coupling strategies applied	287
8.6.1	Heating of a protoplanetary disk	287
8.6.2	Integrating a star cluster with interrupts	290
8.7	Multi-scale modules in AMUSE	292
8.7.1	The <code>multiples</code> module: combining few-body with many-body dynamics	292
8.7.2	The <code>Kira</code> module: combining stellar and binary evolution with stellar dynamics	296
8.7.3	The <code>LonelyPlanets</code> module: combining planetary with stellar dynamics	301
8.8	Validation	305
8.9	Assignments	306
8.9.1	Interlaced timestepping	306
8.9.2	Stellar evolution and dynamics	307
8.9.3	Multiple stellar populations	307
9	Advanced Coupling Strategies	311
9.1	Code-coupling Strategies	311
9.1.1	The Babylonian integrator	313
9.1.2	The Bridge Method	314
9.1.3	Implementation of Bridge	316
9.1.4	Determining the bridge time steps	317
9.2	Using Bridge	318
9.2.1	Star Cluster in a Static Galactic Potential	319
9.2.2	Adding extra functionality with a kick to a bridge	322
9.2.3	The Classic Bridge	326
9.3	Bridging Other Codes	328
9.3.1	Bridge Hierarchies and Hierarchical Bridges	329
9.3.2	Bridging Gravity with Hydrodynamics	333
9.4	Examples	340
9.4.1	Dissolving star cluster in the Galactic potential	340
9.4.2	Did the Sun originate in M67?	345
9.4.3	Inspiral of a binary star into a common envelope	348
9.4.4	Budding planets in a protoplanetary disk	351
9.4.5	Planetary systems in star clusters	351
9.5	Assignments	353
9.5.1	Drift with gravity code	353
9.5.2	How did the Sun escape from M67?	354
9.5.3	Half-treecode, half-direct	356

9.5.4	The accreting black hole in HLX-1	357
9.5.5	The formation of the widest binary stars	358
9.5.6	Photoevaporating disk in binary system	360
10	Expanding and extending AMUSE	366
10.1	Intricate multi-scale coupling strategies	366
10.1.1	Encounters: multiples in stellar clusters	367
10.1.2	Nemesis: planetary systems in stellar clusters	369
10.1.3	Venice: a generalized multi-scale bridge	373
10.2	Advanced coupling projects	375
10.2.1	Torch: forming star clusters from molecular clouds	375
10.2.2	Ekster: forming clusters from gas to star	377
10.3	Supernova impact on the early solar system	378
10.3.1	Initial conditions and model parameters	380
10.3.2	The combined solver	381
10.3.3	Radiative hydrodynamics with cooling and heating	382
10.3.4	Injection of the supernova blast wave	382
10.3.5	The supernova blast wave hits the disk	384
10.4	Accretion in the Galactic center from S-star winds	387
10.4.1	Initial conditions	388
10.4.2	The combined solver	389
10.4.3	Results of the simulation	393
10.5	Visualisation	395
10.5.1	Fresco: visualising gas-rich dusty star clusters	395
10.6	Closing	396
	Epilogue	401
	Appendices	403
	Appendix A AMUSE Fundamentals	404
A.1	Internal parameters, units and types	404
A.1.1	Non-tunable parameters	404
A.1.2	Tunable parameters	405
A.1.3	Input/output file types	410
A.2	Initial Conditions	411
A.2.1	Generalized initial condition generating routines	411
A.2.2	Writing composite datafiles	414
A.3	Notes on specific codes	415
A.3.1	Test particles in N -body codes	415
A.3.2	Post-Newtonian integration with relativistic cross terms	415
A.3.3	The <i>Huayno</i> multi-component symplectic N -body code	416
A.3.4	Symple: The simplest possible symplectic N -body integrator	417
A.3.5	External galactic potential models	419
A.4	Higher-order bridges	422
A.4.1	Fundamentals of high-order bridges	422

A.4.2	Tuning the higher-order bridge	424
A.5	Stopping and boundary conditions	424
A.5.1	Hydrodynamical boundary conditions	425
A.5.2	Stopping conditions	425
A.6	Other handy routines	426
A.6.1	Particles and particle sets	426
A.6.2	Particle attributes	427
A.6.3	Particle queries and functions	429
A.6.4	Grid-based data	431
A.7	Plotting	431
A.7.1	Basic plotting in Python	432
A.7.2	Plotting with units	432
A.7.3	Additional plotting utilities	433
A.7.4	Multiprocessing Codes	434
A.8	AMUSE community modules	436
A.8.1	Modules by domain and functionality	436
A.8.2	Modules by name	436
A.8.3	Packages removed since the 2018 version of the book	454
Appendix B How to add a code to AMUSE		464
B.1	The AMUSE framework	464
B.1.1	The community module	465
B.1.2	The user script	469
B.1.3	Inter-module data transfer and unit conversion	469
B.1.4	Installing prerequisites and debugging	470
B.1.5	Reproducibility with <code>git</code>	471
B.1.6	Tuning your AMUSE installation	472
B.1.7	Debugging in AMUSE	473
B.2	Other associated codes and packages	473
B.2.1	TRES: Triple-star Evolution	474
B.2.2	AGAMA: Action-based Galaxy Modelling Architecture	474
B.2.3	Vader: Viscous Accretion Disk Evolution Resource	476
B.2.4	FRIED: Far-ultraviolet Radiation Induced Evaporation of Discs	477
B.2.5	PACE: Planet ACcretion and Evolution	478
B.2.6	Packages not yet used in scientific production	479
B.2.7	Solar system initial conditions	480
B.2.8	The interface to interstellar non-equilibrium chemistry	480
B.2.9	Tidymess: TIdal DYnamics of Multi-body ExtraSolar Systems	483
Appendix C Programming Primer		488
C.1	Linux	488
C.1.1	A little <code>awk</code> ing and <code>grep</code> ping	490
C.2	Git	491
C.3	Python	492
C.3.1	Python types and printing	493

C.3.2	Lists	494
C.3.3	Dictionaries	495
C.3.4	Flow control	496
C.3.5	Functions	497
C.3.6	Importing packages	498
C.3.7	Classes	498
C.3.8	Operator overloading	499
C.3.9	Properties	500
C.3.10	Testing and debugging Python scripts	501

List of Figures

1.1	Workflow for designing a production script.	10
1.2	Stela of Senenmut's tomb.	14
1.3	Clay tablet number BM40054 with cuneiform writing.	15
1.4	Initial configuration for a simulated Galaxy.	16
1.5	Primitive simulation of a Galaxy merger.	16
1.6	Fundamental hierarchies in complex astronomical simulations.	18
1.7	AMUSE schematics.	20
1.8	General AMUSE structure.	22
2.1	Particle distribution of a projection gas filament.	36
2.2	Various grid-based data representations.	36
3.1	Common initial particle distributions.	51
3.2	Initial density profiles for an SPH code.	55
3.3	Initial mass function.	56
3.4	Orbits of the Sun, Venus and Earth.	61
4.1	Globular cluster M80.	75
4.2	Dynamical evolution of a simple two-component stellar system.	76
4.3	Workflow of an N -body code.	80
4.4	Wall-clock time for several N -body codes.	94
4.5	Evolution of Earth's orbit.	98
4.6	Orbits of Venus, Earth and Mars.	99
4.7	Vostok temperature measurements.	100
4.8	Montgomery problem in GR	101
4.9	3-body orbit with Brutus	103
4.10	3-body phase-space distance	104
4.11	Secular evolution of a hierarchical quadruple system.	106
4.12	Projected view of two galaxies colliding.	108
4.13	Energy error of an N -body integration.	112
4.14	Equipotential surface of a two-body system.	115
4.15	Evolution of the virial ratio of a star cluster.	116
5.1	Observation of stellar surface	125
5.2	Schematic stellar structure.	125
5.3	Hertzsprung–Russell diagrams for several stars	126

5.4	Stellar structure as a function of luminosity.	130
5.5	Surface image of the Sun	134
5.6	Evolution of a star's central temperature and density.	135
5.7	Stellar density profile.	139
5.8	Hertzsprung–Russell diagram for two different stellar-evolution codes	144
5.9	Stellar response to mass loss.	161
5.10	Hertzsprung–Russell diagram for the star cluster M67.	162
5.11	Comparison of the Sun for four stellar evolution codes.	163
6.1	Shock geometry	176
6.2	Kelvin-Helmholtz instability observed in the atmosphere of Jupiter.	180
6.3	Mid-plane density distribution of a $4 M_{\odot}$ star.	190
6.4	Projected density distribution of a star just before a supernova.	191
6.5	Views of the disk with a bump after 2000 yr.	200
6.6	Recomputation of Figure 10 in Benz & Hills (1987).	201
6.7	Accretion rate onto a $1 M_{\odot}$ star from the wind of a $2 M_{\odot}$ companion.	202
6.8	Evolution of the energy of an exploding $10 M_{\odot}$ star.	204
6.9	Collapse of a homogeneous $3 \text{ pc } 10^4 M_{\odot}$ gas cloud.	205
6.10	Mid-plane slice through a collapsing molecular cloud.	209
6.11	Projected density profile of a simulated molecular cloud.	210
6.12	Intensity of emission from the molecule N_2H^+	211
6.13	Riemann shock tube test.	213
6.14	Kelvin-Helmholtz comparison with three grid-based hydro codes.	214
6.15	Cloud-shock test for 3 grid-based hydrodynamics codes.	215
6.16	Boss–Bodenheimer test for a $1 M_{\odot}$ spherical cloud.	216
6.17	Convergence test for merger simulation of a $10 M_{\odot}$ and $1 M_{\odot}$ star.	217
6.18	Evolutionary track for a naked helium star and its progenitor.	222
7.1	Workflow for a radiative transfer code.	235
7.2	Ionization due to a $100 L_{\odot}$ star.	241
7.3	Temperature of the gas distribution around the $100 L_{\odot}$ star.	242
7.4	<code>radiative_h2cloud</code> : Radial electron temperature profile of a centrally illuminated H_2 cloud.	245
7.5	Expansion of the ionization front in a homogeneous medium.	246
8.1	Clumpiness in a 1000-body fractal cluster.	264
8.2	SPH realization of the density profile of a single star	267
8.3	Lagrangian radii for a 1000-star cluster for various mass-loss schemes.	273
8.4	Evolution of orbital parameter for the triple θ Muscae.	275
8.5	Evolution of the Wolf-Rayet triple θ Muscae	276
8.6	Small star cluster with collisions.	278
8.7	Evolutionary tracks of stars and merger product (1).	280
8.8	Projected density distribution of a stellar merger.	284
8.9	Evolutionary tracks for stars and merger product (2).	286
8.10	Evolution of luminosity of two supernovae.	288
8.11	Size distribution of circumstellar disks in the Trapezium cluster.	293

8.12	The AMUSE multiples module.	294
8.13	Semimajor axis and eccentricity of binary stars at birth and 10 Myr. . .	300
8.14	LonelyPlanets Stage 1	303
8.15	Second stage calculation for LonelyPlanets	305
9.1	Hierarchy of time and size scales.	312
9.2	Perato curve for reinforcement bridge time-step.	318
9.3	Snapshot of the Arches cluster near the Galactic center.	323
9.4	IMBH spirals into the Galactic center	324
9.5	Star cluster near the Galactic center.	327
9.6	Schematic diagram of Bridge and ph4 instantiations.	331
9.7	Evolution of the energy error in the Sun–Earth–Moon system.	333
9.8	Initial cluster model composed of stars and gas.	339
9.9	Distribution of the solar siblings in the Galaxy.	341
9.10	Current distribution of the solar siblings.	344
9.11	Solar siblings in the Galaxy.	346
9.12	Evolution of the distance between the Sun and the star cluster M67. . .	347
9.13	Maximum stellar radius at the tip of the asymptotic giant branch. . . .	349
9.14	Evolution of the outer orbital separation of ξ Tau.	350
9.15	Young star with planets in a rather flat gaseous disk.	352
9.16	Nemesis energy behavior.	353
9.17	HLX-1 x-ray counts.	357
10.1	Nemesis strategy.	372
10.2	Venice timescale coupling	374
10.3	Supernova feedback in Torch	376
10.4	Young stellar outflow in Torch	378
10.5	Massive star-forming region	379
10.6	View of the solar system 6.4 years after the supernova explosion.	383
10.7	Irradiation temperature of circum-stellar disk	384
10.8	Surface density profile of the protoplanetary disk.	385
10.9	Heating of the protoplanetary disk due to a supernova explosion.	386
10.10	Inclination evolution of the Sun’s planetary disk.	387
10.11	Evolution of mass-loss rates of high-mass stars.	389
10.12	Orbital integration of the S stars.	390
10.13	Evolution of the S stars in the Galactic center.	394
10.14	Mass accretion by the central supermassive black hole.	395
10.15	Projection of a collapsing molecular cloud.	396
A.1	Cluster with stars, planets and debris disks.	413
A.2	2010 Christmas card of Leiden Observatory	434
B.2	Dust-mass vs. distance to cluster center.	477
B.3	Photo-evaporating disks with Vader and the FRIED grid	478
B.4	Simulated planet population.	480
B.5	Molecular cloud with chemistry.	482

B.6 Chemical evolution of a molecular cloud.	483
--	-----

List of Tables

3.1	Cartesian coordinates for the Sun and 8 planets at 5 April 2063.	58
4.1	Overview of N -body codes in AMUSE.	93
4.2	Stopping conditions.	109
5.1	Stellar evolution codes into AMUSE.	137
5.2	Comparison of four stellar evolution codes.	143
5.3	Observed parameters for open star cluster M67.	160
5.4	Observed parameters for nearby bright stars.	164
6.1	Characteristic regimes for hydrodynamics.	178
6.2	Hydrodynamics codes available in AMUSE.	184
6.3	Stopping conditions.	197
7.1	Radiative transfer codes incorporated into AMUSE	234
8.1	Colliding wind triple θ Muscae.	276
9.1	Performance of various Bridge implementations.	332
9.2	Known triples the tertiary star being the most massive.	348
A.1	Stellar classifications in AMUSE.	405
A.2	Tunable and non-tunable parameters for gravity.	406
A.3	Tunable and non-tunable parameters for stellar evolution.	406
A.4	Tunable and non-tunable parameters for hydrodynamics.	408
A.5	Boolean hydrodynamical parameters	409
A.6	Tunable and non-tunable parameters for radiative transfer.	409
A.7	Recognized read/write file formats.	411
A.8	Initial conditions	412
A.9	Extra options for hydrodynamics initial conditions.	414
A.10	Tunable and non-tunable parameters for relativistic gravity.	416
A.11	Integrators available in <code>Hermite-GRX</code>	416
A.12	Integrators available in <code>Huayno</code>	417
A.13	Symple integration order and number of evaluation steps.	419
A.14	List of (semi)analytic background potential models.	420
A.15	Initial parameters for the Milky Way Galaxy in <code>galaxia</code>	421
A.16	Bridge methods identified by the order of their integrator.	425

A.17 List of stopping condition keywords	427
A.18 Particle attributes for gravity.	428
A.19 Particle attributes for stellar evolution.	428
A.20 Particle attributes for hydrodynamics.	428
A.21 Particle attributes for radiative transfer.	428
A.22 Core code selection	437
A.23 Modules in AMUSE	438
B.1 Dependencies needed to compile and run AMUSE.	470
B.2 Debugging directives.	474

List of code examples

2.1	<code>stellar_minimal</code> : Command-line parser.	44
3.1	<code>plot_plummer</code> : Plotting a Plummer sphere.	50
3.2	<code>plot_hydro_density_distributions</code> : Converting 1-D to 3-D.	53
3.3	<code>plot_hydro_density_distributions</code> : Plotting 3-D particle distribution.	54
3.4	<code>plot_hydro_density_distributions</code> : Generating molecular cloud as a particle representation.	54
3.5	<code>salpeter</code> : Plotting the Salpeter mass function.	57
3.6	<code>sun_venus_earth</code> : Initial conditions for Venus and Earth around the Sun.	59
3.7	<code>sun_venus_earth</code> : Plotting orbits of Venus and Earth.	61
3.8	<code>make_binary_star</code> : Generating binary stars.	63
3.9	<code>make_binary_star</code> : Initializing a Kepler orbit.	63
3.10	<code>make_binary_star</code> : Calculating the orbital elements of a binary.	64
4.1	<code>gravity_minimal</code> : Minimal gravity solver.	86
4.2	<code>sun_venus_earth</code> : Integrating Venus and Earth around the Sun.	95
4.3	<code>plot_gravity</code> : Minimal plotting routine.	96
4.4	<code>anim_gravity</code> : Making a simple animation.	97
4.5	<code>montgomery</code> : Initializing a relativistic gravity solver.	101
4.6	<code>hierarchical_quadruple</code> Setting-up a hierarchical binary-binary.	105
5.1	<code>calculate_core_temperature_density</code> : Calculating stellar core tem- perature and density.	135
5.2	<code>initialize_single_star</code> : Initializing a zero-age main-sequence star.	138
5.3	<code>stellar_minimal</code> : Minimal stellar evolution solver.	140
5.4	<code>multiple_stellar_code</code> : Compare stellar evolution codes.	145
5.5	<code>merge_two_stars</code> : Merging two stars.	148
5.6	<code>binary_evolution</code> : Initializing a stellar binary.	154
5.7	<code>binary_evolution</code> : Evolving a single binary.	155
5.8	<code>massloss_response</code> : Calculating the stellar adiabatic index.	159
5.9	<code>massloss_response</code> : Evolving a star of mass M to the AGB.	160
6.1	<code>hydro_minimal</code> : Minimal hydrodynamics solver.	185
6.2	<code>hydro_outflow_particles</code> : Generating a stellar wind in SPH.	193
6.3	<code>hydro_outflow_particles</code> : Determine wind velocity for SPH particles.	193
6.4	<code>hydro_sink_particles</code> : Sink particle treatment.	195
6.5	<code>hydro_disk_with_bump</code> : Generating a disk with a bump.	198
6.6	<code>explode_evolved_star</code> : A supernova explosion in SPH.	203
6.7	<code>helium_star</code> : How to strip a stellar envelope.	221
7.1	<code>rad_minimal</code> : Minimal radiative transfer routine.	237

7.2	<code>rad_minimal</code> : Generating a scattering gas medium.	238
7.3	<code>radiative_h2cloud</code> : Initializing a uniform density grid.	241
7.4	<code>radiative_h2cloud</code> : Initialize a radiative transfer code.	243
7.5	<code>radiative_h2cloud</code> : Event loop for radiative transfer.	245
8.1	<code>gravity_stellar_minimal</code> : Combining stellar evolution with dynamics.	254
8.2	<code>gravity_stellar_minimal</code> : Event loop for stellar evolution and gravity.	255
8.3	<code>gravity_stellar_collision</code> : Simulating a cluster with stellar collisions.	261
8.4	<code>gravity_stellar_collision</code> : Resolving stellar collisions.	262
8.5	<code>gravity_stellar_collision</code> : Merge two stars and rejuvenate.	262
8.6	<code>plot_fractal_clumpy_cluster</code> : Finding clumps in a particle distribution.	263
8.7	<code>gravity_stellar_eventdriven</code> : Recovering the internal stellar structure.	270
8.8	<code>gravity_stellar_eventdriven</code> : Event-driven simulation of a star cluster.	271
8.9	<code>gravity_stellar_eventdriven</code> : Resolving the supernova in a stellar cluster.	272
8.10	<code>merge_two_stars_and_evolve</code> : Merge two stars and continue the evolution.	278
8.11	<code>merge_two_stars_sph</code> : Construct a SPH realization from a stellar model.	281
8.12	<code>merge_two_stars_sph</code> : Merge two stars with SPH.	282
8.13	<code>merge_two_stars_sph</code> : Relax (glasify) a non-equilibrium SPH model.	283
8.14	<code>gravity_kepler_disks</code> : Using interrupts in an N-body simulation.	290
8.15	<code>gravity_kepler_disks</code> : Resolving star-disk encounters.	291
8.16	<code>gravity_kepler_disks</code> : Truncating a circum-stellar disk.	292
8.17	<code>basic_multiples</code> : A simple N-body integration with close encounter treatment.	295
8.18	<code>basic_multiples</code> : Initialize the child N-body code in multiples.	296
8.19	<code>kira</code> : Adapting binary in <code>kira.py</code>	299
8.20	<code>lonely_planets</code> : stage 1.	301
8.21	<code>lonely_planets</code> : stage 2.	304
9.1	<code>gravity_potential</code> : Integrating in a gravitational potential.	319
9.2	<code>gravity_potential</code> : Initializing the bridge for gravity with potential.	320
9.3	<code>gravity_potential</code> : Bridging gravity code with potential.	322
9.4	<code>imbhs_with_friction</code> : Augmented bridge.	325
9.5	<code>gravity_drifter</code> : Drifter code for bridge.	328
9.6	<code>two_body_bridge</code> : Bridging gravity for Sun-Earth system.	329
9.7	<code>three_body_bridge</code> : Multi-bridge for Sun, Earth and Moon.	330
9.8	<code>three_body_bridge_hierarchical</code> : Hierarchical bridge for Sun, Earth and Moon.	331
9.9	<code>gravity_hydro</code> : Bridging gravity with hydrodynamics.	335
9.10	<code>gravity_hydro</code> : Generic bridge base class.	336
9.11	<code>gravity_hydro</code> : Derived class for gravity.	336
9.12	<code>gravity_hydro</code> : Derived class for hydrodynamics.	336
9.13	<code>relax_gas_and_stars</code> : Using bridge for relaxing a gaseous cloud.	338
9.14	<code>relax_gas_and_stars</code> : Monitoring energy conservation while bridging.	338

9.15	<code>evolve_xi_tau_primary</code> : Turning a stellar model in an SPH realization.	359
10.1	<code>multiples_minimal</code> : Initializing binaries for the multiples module. . . .	368
10.2	<code>multiples_minimal</code> : Initializing the multiples module.	368
10.3	<code>multiples_minimal</code> : Handing particles to the multiples module.	369
10.4	<code>kira</code> : Setting up the encounters module.	370
A.1	<code>multiple_stellar_threaded</code> : Evolving a single star.	435
A.2	Concurrent operation.	435
B.1	Automated download script.	471
B.2	<code>tres_minimal</code> : Minimal example of <code>Tres</code>	475
C.1	<code>class_example</code> : A base class	498
C.2	<code>class_example</code> : Derived class fragments	499
C.3	<code>class_example</code> : A compound class	499

Preface

This is the second, revised, edition of the academic textbook on the Astrophysics Multipurpose Software Environment, or AMUSE, for short ([Portegies Zwart & McMillan, 2018](#)). The AMUSE project started at the MODEST-5 conference in Hamilton, Ontario, in August 2004. Steve McMillan and Simon Portegies Zwart were sipping coffee, nibbling on cookies, and discussing the future of **starlab**.

Starlab began in the early 1990s as an alternative and competitor to Sverre Aarseth’s NBODY series of codes. In collaboration with Piet Hut and Jun Makino, we wrote a modular package combining stellar dynamics and evolution using modern programming paradigms and data structures. The code was written in C++, an up-and-coming language at the time, which afforded many opportunities to go beyond the rigid arrays used in existing languages at the time.

The **starlab** package contained many novel features not found in comparable state-of-the-art codes even today. Most codes at the time, and certainly the leading NBODY series developed by Sverre Aarseth, were “monolithic” solvers, designed, written, and integrated for one particular purpose. A direct summation N -body code with a hard-coded realization of stellar evolution is an excellent example of a monolithic code. Because monolithic codes are written in a single programming language with simple data structures, they are generally relatively uncomplicated, straightforward to compile, and quite portable.

By contrast, **starlab** was a non-monolithic code composed of two independent components; a gravitational N -body solver **kira** and the binary population synthesis code **SeBa**. Much of the internal coding in **starlab** was devoted to maintaining consistent communication between its constituent parts. The result was still quite tightly coupled, but it preserved the essential kernel of independent communicating parts. But as we contemplated expanding our scope by including hydrodynamics, we realized that, although we had anticipated this possible expansion, we still did not really know how to do it without radically changing the basic data structures and rewriting the code. Hard-coding a hydro module into **starlab** would be counter to all our design choices and would be horribly complicated to code and maintain, but how else might we expand the code’s functionality?

After more than a decade of reliable service and hundreds of scientific papers, **starlab**, we realized, faced complicated design choices for the future expansion of the package. Or should we start all over again?

We had no clear solution to the problem, and we felt insufficiently confident to start all over until Sverre Aarseth and Peter Eggleton joined us. We started talking about the codes they had written. Sverre initiated and led the NBODY series of direct

summation N -body solvers. His codes are highly optimized and always written in a historic FORTRAN dialect. His latest incarnation NBODY6, at that time, was the old-school counterpart of `starlab`. NBODY6 was faster than `starlab`, but (we think) was less elegantly structured, if one can speak of any formal structure. Peter is a pioneer in writing stellar structure and evolution codes. The time Sverre spent writing N -body codes, Peter spent improving `EVTwin`, his stellar evolution code, also in vintage FORTRAN.

Sverre looked very happy to hear that we considered `starlab` past its prime. “NBODY6 is brand new” he told us, “very alive, and kicking bodies, yes.” We thought that, in some way, we might benefit from the many decades of combined experience of these two grandfathers of computational astrophysics. Why redesign code? Why rewrite any software if we already have the right tools at hand? If only they would turn into useful tools with a proper interface. Could it be possible to incorporate one of these codes into the other? We asked Sverre how he would feel about this. He responded: “Wouldn’t it be nice to have Peter’s stellar evolution code as a part of my beautiful N -body code?” Peter responded: “But, Sverre, my dear friend, how splendid would it be to have your N -body code as part of my stellar evolution code.”

That got us thinking. Maybe we should get away from “mine” and “thine,” and operate instead in a collective framework, and we proposed: “What about if both your codes could work together?” While hardly ever in agreement, both men thought this would be an interesting idea, like the Borg (of whom we suspect neither had ever heard) assimilating the positive characteristics of everybody to become better as a whole. But they were even more sure that such an attempt would be impossible. You would have codes written in all sorts of languages. How would you deal with all the nasty communications and unit conversions worldwide? Also the communication between codes will become a major bottleneck. No, better to have another cookie and dream about NBODY7.

By now, NBODY7 turned into NBODY6++; still in FORTRAN, though. But in that conversation, we had the kernel needed to get started; together, we can rule the Galaxy. That evening, we had a quiet dinner and discussed the possibilities of a language-independent framework in which codes would operate as modules that carry out specific tasks. The communication between the various codes would have to go through some sort of interface, to be determined, allowing it to perform operations on the data, formatting it to the requirements of the other codes, and scheduling the execution of those codes. Maybe it was not so impossible after all.

The MODEST initiative, which gave rise to the Hamilton conference where this exchange took place, was a direct outgrowth of our experience with `starlab`. However, it eventually became clear that, even with the best intentions, we still had combined algorithms in a way that made it hard to make radical changes to the fundamentals.

This realization led Piet Hut and Mike Shara in 2001 to organize a conference at the American Museum of Natural History in New York, to bring together experts in stellar dynamics, stellar evolution, stellar hydrodynamics, computer science, and other relevant areas, to seek truly modular ways to combine codes to create a new computational framework. The meeting was subsequently named MODEST-1 (for MOdeling DENSE STellar systems). Perhaps unsurprisingly, the early sample codes that emerged from

the meeting looked very much like `starlab`, but MODEST (subsequently re-acronymed as Modeling and Observing DENSE STellar systems) rapidly became an important forum for the discussion of all aspects of dynamical modeling and observations of dense stellar systems.

Early MODEST meetings focused more on astrophysics than computation. Still, following the MODEST-5 meeting in Hamilton, a rapid series of satellite meetings changed the software landscape. We organized a meeting in Lund, Sweden, in December 2005 and formulated a concrete plan for combining existing codes. The discussion continued a few months later in Amsterdam, where we organized another small workshop to discuss the possibilities. Piet Hut was there, along with Jun Makino, Peter Teuben, Pierro Spinnato, and Breannndán Ó Nualláin. Breannndán was a Python aficionado, and with his help, we had the first version of a rudimentary working code operational in a few days. We called this framework MUSE, for Multipurpose Software Environment.

The key idea of MUSE was that no code got to “drive” the simulation. Dynamical, stellar evolution and other codes were invoked as modules, with suitably defined interfaces, but the top-level code was a Python script that created and coordinated the scientific study. The implementation involved heavy use of Python tools such as `f2py` and `swig`, and ran into numerous practical problems that needn’t concern us here, but the basic idea of a script-driven, democratic code framework was established.

After the fundamental ideas had been put into practice, and once some of their shortcomings were recognized, we realized that a bunch of astronomers would not be able to make quick progress on such a large software environment. We searched for funding to hire a dedicated software architect, Arjen van Elteren, who could help design our framework, and two computational astronomers, Inti Pelupessy and Nathan de Vries, to develop the first science cases with the new framework. The team turned out to be very effective. The framework is now called AMUSE, for Astrophysical MULTipurpose Software Environment. Over the last decade we worked hard with students to improve the package, broaden its scope and widen the scientific applications. The most recent developments include incorporating codes for simulating exo-planetary atmospheres and interstellar chemistry, and the entire build-system was renewed. This manuscript explains some of the functionality and operations possible with the framework.

The main objective of this book and the entire AMUSE framework is to explore new science. To seek out new phenomena and new discoveries. To boldly go where no computer has gone before! This is the continuing mission of this exciting journey. The first edition of the AMUSE book, published in 2018, reflected the authors’ roots in simulations of dense stellar systems, as well as the historical development of the field. In this new version, we sought new collaborator and found him in Steven Rieder. He was closely connected to the AMUSE endeavor from the beginning; somewhat junior at the time of the initial development, he has developed to a true AMUSE master, and took care of most of the maintenance, documentation, updates, bug fixes, et cetera; AMUSE’s presence is strong in our academic family¹.

¹Adopted from Luke Skywalker to Leia Organa in Star Wars: Return of the Jedi (1983).

The new book is divided into three parts. In part I, we discuss the fundamentals of AMUSE and the working of the monolithic solvers; gravity, stellar evolution, hydrodynamics, and radiative transfer. In part II, we discuss elementary coupling strategies adopted in AMUSE. Part III brings all AMUSE capabilities together and shows the reader how to design complicated multi-scale and multi-physics experiments. In the last part, we also show how AMUSE can be adapted and applied to an even broader range of problems. We present complete AMUSE scripts for performing high-fidelity simulations and provide some examples outside the domain of astrophysics.

But above all, the goal of this book is to have fun. To enjoy computational astrophysics, and to explore the nonlinear couplings between different physical domains without having to worry too much about the details of energy conservation in a close three-body interaction or managing the arcana of high-dimensional differential equations. Decades of computational astrophysics research have gone into some of the modules incorporated into AMUSE. These are trustworthy codes and form an excellent basis for further study. Still, never trust a code that doesn't work, but trust a working code even less.

The AMUSE framework is still a work in progress. Even after more than a decade of coding, thinking, and writing, we suspect this book and the framework to contain errors and typos that neither we, our reviewers, nor our editors caught. For these reasons, we will maintain an evolving document in AMUSE that lists corrections to the published text and highlights developments in AMUSE that may interest readers, such as new algorithms, functionality, or insights into the many problems we address. It can be found in `{AMUSE_DIR}/examples/textbook/updates/AMUSE_updates.pdf`, which can be saved via

```
from amuse.examples import export
export("AMUSE_updates.pdf", savedir=".")
```

where `amuse.examples` is the examples package companion to this book (see Section 1.4). Incidentally, the “laptop” cited in the timing estimates for most figures is Simon’s Lenovo P14s, 20-core 3th Gen Intel Core i7-1370P with NVIDIA RTX A500 GPU. Your timing may differ.

Simon Portegies Zwart
Steve McMillan
Steven Rieder

Bibliography

Portegies Zwart, Simon, & McMillan, Steve. 2018. *Astrophysical Recipes*. 2514-3433. IOP Publishing.

Acknowledgments

I don't know half of you half as well as I should like; and I like less than half of you half as well as you deserve.

J.R.R. Tolkien

These words were uttered by Bilbo Baggins on 22 September 3001 in front of 144 invited guests to celebrate his eleventy-first birthday. With this speech he wanted to say farewell to and acknowledge friends and family. We also would like to acknowledge a large number of people who contributed to this book, although we will not go so far as to insult half of them. We at least do not consider any of them at the same level as Otho and Lobelia.

AMUSE is really a community effort, and a large number of people have contributed. The master students at Leiden Observatory helped eliminating bugs and typos during the various courses on computational astrophysics. We apologize to them for having used them as guinea pigs for the first version of the manuscript². We particularly thank the many people who contributed by adding new code, developing or debugging scripts, or initiating new ideas through stimulating discussions. They include: Assilkan Adilkhan, Eric Andersson, Sabrina Appel, Jeroen Bédorf, Tjarda Boekholt, Anthony Brown, Maxwell Cai, Claude Cournoyer-Cloutier, Niels Drost, Will Farnar, Michiko Fujii, Evghenii Gaburov, Joseph Glaser, Guilherme Goncalves, Adrian Hamers, Erwan Hochart, Shuo Huang, Piet Hut, Masaki Iwasawa, Lucie Jílková, Nadia Kempers, Rony Keppens, Ralph Klessen, Rob Knop, Jun Makino, Tunde Meyer, Sean Lewis, Carmen Martínez-Barbosa, Mordecai-Mark Mac Low, Breannán Ó'Nualláin, Jan-Pieter Paardekooper, Veronica Saz Ulibarrena, Alison Sills, Jelmer Stroe, Andrew Pellegrino, Brooke Polak, Silvia Toonen, Aaron Tran, Alessandro Trani, Gijs Vermariën, Serena Viti, Joshua Wall, Long Wang, Julia Wasala, Maite Wilhelm, Alfred Whitehead, and Thomas Wijnen.

Several of the nicest figures were made by Edwin van der Helm (Figure 8.5), Alex Rimoldi (Figure 6.4), Nora Lützgendorf (Figure 10.13), and Sabria Apple (Figure 10.4).

We received illuminating private lectures on gravitational dynamics (Chapter 4) by Sverre Aarseth, on stellar evolution (Chapter 5) by Onno Pols and Peter Eggleton, on hydrodynamics (Chapter 6) by James Lombardi and Volker Springel, and on radiative transfer (Chapter 7) by Barbara Ercolano and Kees Dullemond.

During the development of AMUSE and the writing of the book we made extensive use of many additional public facilities and packages, including: ADS, arXiv, CDS, git,

²Here we refer to the *Cavia porcellus*, in honor of the 1901 Nobel prize in Medicine, awarded to E. Adolf von Behring, who used this cute furry rodent to develop a serum therapy against diphtheria.

hdf5, ipython, Jupyter, L^AT_EX, linux, MPI, matplotlib, numpy, OpenMP. python, Seaborn, wikipedia, and probably many more we forgot to mention.

Both authors had the opportunity to spend an extensive amount of time on this book, for which we are grateful to our (former) host institutions, Leiden Observatory, Drexel University, Geneva Observatory, KU Leuven and the Anton Pannekoek Institute for Astronomy. But we are also grateful to the Center for Planetary Science in Kobe, the Lorentz Center in Leiden, Geneva Observatory, the University of Strasbourg, the Anton Pannekoek Institute, Universiteit Leuven for hosting our discussions. Parts of the development of AMUSE were financed by NOVA NWO (639.073.803 and 614.061.608), NASA (grants NNX07AG95G, NNX08AH15G), NSF (AST0708299), H2020 FET Com-Pat, the EU P7/08 CHARM project and the NLeSc. We used compious amounts of Supercomputer time on SURF/Snellius (2023.008, 2024.058, EINF-10097, EINF-11229, EINF-12718, EINF-13923) and LUMI (EINF-11368).

Without Arjen van Elteren, Inti Pelupessy, and Nathan de Vries, AMUSE would not exist, and we would never have been able to write this book. A special thank you also is due to the anonymous referee Douglas Heggie (of the first version), who identified many omissions and errors, which we hope we have corrected. We also thank our editor and staff at IoP and in particular Robbie Trevellian, Leigh Jenkins, and Daniel Heatley for their support of this writing project.

Our spouses and children earn considerable praise for their patience. They are very happy that this is over, and we are happy that they are still there.